

Python基础编程与工具开发培训班

第一期学员名单

行政事务部/党群部 高雯欣

财务部 胡玥

单证中心 邓若妍

订舱中心 鄯雅宁

订舱中心 吴范淼

支持中心 王彤飞

支持中心 徐缘

审核中心 张杰君

审核中心 邱兴源

大连分公司 郭晶

大连分公司 姚虎城

大连分公司 罗冉

大连分公司 何进

武汉分公司 何谦

武汉分公司 高娟

武汉分公司 胡天意

武汉分公司 张丹丹

Python 编程基础教程（完整版）

目录

1. Python 概述与环境搭建	5
核心知识点.....	5
2. 变量与基本数据类型.....	6
2.1 变量命名规则.....	6
2.2 基本数据类型.....	6
2.3 类型转换.....	6
# 2. 整数转浮点数.....	7
# 3. 任意类型转字符串.....	7
3. 输入输出操作.....	7
3.1 输出函数：print()	7
3.2 输入函数：input().....	8
4. 流程控制结构.....	8
4.1 选择结构（if-elif-else）	8
4.2 循环结构.....	9
4.2.1 for 循环（遍历序列）	9
4.2.2 while 循环（条件控制）	10
4.2.3 循环控制：break 与 continue	10
5. 核心数据结构（含元组）	10
5.1 序列类型对比（列表 vs 元组）	11
5.2 元组核心操作.....	11
5.3 其他核心数据结构.....	12
5.3.1 字典（dict）：键值对存储.....	12
5.3.2 集合（set）：无序去重.....	13
# 4. 添加/删除元素	13
6. 函数定义与应用.....	13
6.1 函数基础语法.....	13
6.2 函数参数类型.....	14
6.2.1 位置参数（默认）	15
6.2.2 关键字参数.....	15
6.2.3 默认参数.....	15
6.2.4 可变参数（*args）	16
6.2.5 关键字可变参数（**kwargs）	16
7. 面向对象编程基础.....	16
7.1 类与对象的基本概念.....	16
7.2 定义类与创建对象.....	17
7.3 继承特性（子类与父类）	17
8. 文件操作（文本 / 二进制文件）	18

8.1 文本文件操作.....	18
8.1.1 写入文本文件.....	19
8.1.2 读取文本文件.....	19
8.2 二进制文件操作.....	20
8.3 常用文件打开模式.....	20
9. 模块与包（import 使用）.....	21
9.1 模块（Module）.....	21
9.1.1 导入模块的 4 种方式.....	21
9.1.2 自定义模块.....	22
9.2 包（Package）.....	23
9.2.1 包的目录结构.....	23
9.2.2 导入包中的模块.....	23
9.2.3 __init__.py 的作用.....	24
10. 异常处理（try-except）.....	24
10.1 异常处理基础语法.....	24
10.2 常见异常类型.....	25
10.3 自定义异常.....	26
11. 正则表达式基础.....	27
11.1 核心元字符与量词.....	27
11.2 re 模块常用函数.....	29
11.2.1 re.match(pattern, string).....	29
11.2.2 re.search(pattern, string).....	30
11.2.3 re.findall(pattern, string).....	30
11.2.4 re.sub(pattern, repl, string).....	31
11.3 实用场景示例：验证邮箱格式.....	31
12. 列表推导式与字典推导式.....	32
12.1 列表推导式.....	32
12.1.1 基础语法.....	32
12.1.2 带条件筛选的列表推导式.....	33
12.1.3 嵌套列表推导式.....	33
12.2 字典推导式.....	34
12.2.1 基础语法.....	34
12.2.2 带条件筛选的字典推导式.....	34
12.2.3 用两个可迭代对象生成字典.....	35
13. 综合实战案例：学生信息管理系统.....	35
13.1 完整代码.....	35
13.2 代码说明与运行结果.....	40
13.2.1 核心知识点整合.....	40
13.2.2 运行结果.....	40
14. 知识点总览与拓展.....	41
14.1 核心知识点全覆盖（分层梳理）.....	41

14.2 拓展学习方向.....	42
15. 结束页	错误!未定义书签。

章节	学习目标
1. Python 概述 与环境搭建	1. 了解 Python 语言的核心特点（简洁、动态类型、跨平台）及典型应用场景；2. 掌握 Python 解释器的下载安装（含 PATH 配置）及主流 IDE（PyCharm/VS Code/Jupyter）的使用；3. 能够编写并运行第一个 Python 程序（Hello World），熟悉基础运行流程。
2. 变量与基本数据类型	1. 牢记变量命名规则（字母 / 数字 / 下划线、不数字开头、区分大小写、避开关键字）及蛇形命名法；2. 掌握 4 种基本数据类型（int/float/str/bool）的定义与特性；3. 熟练运用类型转换函数（int ()/float ()/str ()），理解转换逻辑与异常场景。
3. 输入输出操作	1. 掌握 print () 函数的核心用法（多参数、f-string 格式化、sep/end 参数自定义）；2. 理解 input () 函数的默认返回值类型，能根据需求转换输入数据类型（int/float）；3. 能够实现用户输入与结果输出的交互逻辑。
4. 流程控制结构	1. 熟练使用 if-elif-else 选择结构，掌握冒号与缩进的语法规则；2. 掌握 for 循环（遍历序列、range () 函数）与 while 循环（条件控制）的适用场景；3. 理解 break（终止循环）与 continue（跳过当前循环）的作用，避免死循环。
5. 核心数据结构 （含元组）	1. 对比列表（可变）与元组（不可变）的差异，明确各自适用场景；2. 掌握列表 / 元组的索引、切片、增删改查操作，字典的键值对访问与修改，集合的去重与集合运算；3. 理解元组拆包特性，能灵活运用四种数据结构存储组织数据。
6. 函数定义与应用	1. 掌握函数的基础语法（def 关键字、参数列表、返回值、文档字符串）；2. 熟练运用 5 种参数类型（位置参数、关键字参数、默认参数、*args、**kwargs）；3. 能够自定义函数实现特定功能，并通过调用函数复用代码。

章节	学习目标
7. 面向对象编程基础	1. 理解类（抽象模板）与对象（具体实例）的核心概念；2. 掌握类的定义（__init__构造方法、实例属性、实例方法）与对象的创建；3. 理解继承特性，能定义子类并复用父类属性方法，新增子类专属功能。
8. 文件操作（文本 / 二进制文件）	1. 掌握文本文件的读写操作（write()/read()/readline()/readlines()）及编码设置（utf-8）；2. 理解二进制文件的操作逻辑（rb/wb 模式），能实现文件复制等功能；3. 牢记常用文件打开模式（r/w/a/r+/wb 等）的特性（是否覆盖、是否创建新文件）。
9. 模块与包（import 使用）	1. 掌握导入模块的 4 种方式（import/from...import/别名 /*）及适用场景；2. 能够编写自定义模块并导入使用，理解包的目录结构与__init__.py 的作用；3. 学会导入包中模块的多种方式，实现代码的模块化组织。
10. 异常处理（try-except）	1. 掌握 try-except-else-finally 的基础语法，能捕获常见异常（ValueError/TypeError 等）；2. 了解常见异常类型的触发场景，能通过异常对象获取错误信息；3. 能够自定义异常类，并用 raise 语句主动触发异常。
11. 正则表达式基础	1. 掌握核心元字符（.^\$[]\d\w\s）与量词（+?{n,m}）的含义；2. 熟练使用 re 模块常用函数（match/search/findall/sub）实现字符串匹配、查找、替换；3. 能运用正则表达式验证特定格式（如邮箱）。
12. 列表推导式与字典推导式	1. 掌握列表推导式的基础语法、条件筛选与嵌套用法，替代繁琐 for 循环；2. 学会字典推导式的基础用法、条件筛选及基于两个可迭代对象的生成方式；3. 能够用推导式简洁高效地生成列表与字典。
13. 综合实战案例：学生信息管理系统	1. 整合面向对象、文件操作、异常处理、正则表达式等核心知识点；2. 理解系统的核心逻辑（加载历史信息、添加学生、筛选学生、保存信息）；3. 学会在实战中处理边界情况（学号格式验证、重复学号、文件不存在等）。
14. 知识点总览	1. 梳理 Python 核心知识点的层级结构（基础层、数

章节	学习目标
与拓展	据结构层等），构建知识体系；2. 了解基础后的拓展学习方向（高级数据结构、网络编程、数据处理等）；3. 掌握推荐学习资源的使用方法，为后续深入学习铺垫。

1. Python 概述与环境搭建

核心知识点

🔗 语言特点：简洁易学、动态类型（无需提前声明变量类型）、跨平台（Windows/macOS/Linux 均支持）。

🔗 应用场景：数据处理（Excel 自动化、日志分析）、自动化脚本（文件批量处理）、Web 开发（Django/Flask 框架）、人工智能（TensorFlow/PyTorch 基础）等。

- 🔗 环境搭建：
- i. 下载 Python 解释器：访问 [Python 官网](#)，选择对应系统版本（建议 3.8 及以上），安装时勾选 “Add Python to PATH”（便于命令行调用）。
 - ii. 推荐 IDE（集成开发环境）：
 - PyCharm（专业版功能全，社区版免费，适合项目开发）；
 - VS Code（轻量，需安装 Python 插件，适合快速编写）；
 - Jupyter Notebook（交互式编程，适合数据处理与教学演示）。

🔗 第一个程序（Hello World）：

```
python
运行
# 单行注释:这是 Python 的第一个程序 print("Hello, Python!") # 输出函数:
在控制台打印指定内容

运行结果：控制台输出 Hello, Python!
```

2. 变量与基本数据类型

2.1 变量命名规则

由字母（a-z/A-Z）、数字（0-9）、下划线（_）组成，不能以数字开头（如 1name 错误，name1 正确）。

区分大小写（name 与 Name 是两个不同变量）。

禁止使用 Python 关键字（如 if、for、while、class 等，关键字会被 IDE 标红提示）。

建议命名风格：使用“蛇形命名法”（全小写，单词间用下划线连接，如 student_age），见名知意。

2.2 基本数据类型

类型	示例	说明
整数（int）	age = 25	无长度限制，可表示正负整数（如-100、999999999999）
浮点数（float）	height = 1.75	表示小数或科学计数法（如 3.14、1.2e3 即 1200.0），注意浮点数有精度误差（如 0.1+0.2≠0.3）
字符串（str）	name = "Alice"	用单引号（' '）、双引号（" "）或三引号（''' '''/" "" "" ""）包裹，三引号支持多行文本
布尔值（bool）	is_student = True	仅两个取值：True（真）或 False（假），常用于条件判断

2.3 类型转换

当需要不同类型的数据参与运算时，需手动转换类型，常用转换函数如下：

python

运行

1. 字符串转整数

```
num_str = "123"
```

```
num_int = int(num_str) # 转换后 num_int 为 123 (int 类型)
```

2. 整数转浮点数

```
num_float = float(num_int) # 转换后 num_float 为 123.0 (float 类型)
```

3. 任意类型转字符串

```
num_str2 = str(num_float) # 转换后 num_str2 为 "123.0" (str 类型)
```

```
# 验证转换结果 print(type(num_int))          # 输出: <class  
'int'>print(type(num_float)) # 输出: <class 'float'>print(type(num_str2)) # 输出:  
<class 'str'>
```

⚠ 注意：若转换不符合逻辑（如 `int("abc")`），会触发 `ValueError`（后续“异常处理”章节会解决此问题）。

3. 输入输出操作

3.1 输出函数：print()

用于在控制台打印内容，支持多参数、格式化输出等功能：

python

运行

```
# 1. 基本输出：多参数用逗号分隔，默认以空格拼接 print("姓名: ", "张三",  
", "年龄: ", 20) # 输出: 姓名: 张三 年龄: 20
```

```
# 2. 格式化输出（推荐 f-string, Python 3.6+支持）
```

```
name = "李四"
```

```
score = 95.5print(f"姓名: {name}, 成绩: {score}") # 输出: 姓名: 李四, 成  
绩: 95.5# f-string 中可直接写表达式（如计算）print(f"成绩的 2 倍: {score * 2}")  
# 输出: 成绩的 2 倍: 191.0
```

3. 自定义分隔符（用 `sep` 参数）`print("a", "b", "c", sep="-")` # 输出：a-b-c

4. 自定义结束符（默认是换行`\n`，用 `end` 参数修改）`print("第一行", end=";")`
`print("第二行")` # 输出：第一行；第二行

3.2 输入函数：input()

用于获取用户从控制台输入的内容，默认返回字符串类型，需根据需求转换类型：

python

运行

1. 获取字符串输入（无需转换）

`username = input("请输入用户名：")` # 若用户输入"Alice"，`username` 为"Alice"（str 类型）

2. 获取整数输入（需转换为 int）

`age = int(input("请输入年龄："))` # 若用户输入"20"，转换后 `age` 为 20（int 类型）

3. 获取浮点数输入（需转换为 float）

`height = float(input("请输入身高（米）："))` # 若用户输入"1.75"，转换后 `height` 为 1.75（float 类型）

4. 输出用户输入的内容 `print(f'欢迎你，{username}（{age} 岁，身高{height} 米）')`

4. 流程控制结构

流程控制用于控制代码的执行顺序，核心包括“选择结构”和“循环结构”。

4.1 选择结构（if-elif-else）

根据条件判断执行不同代码块，语法中冒号(:) 和缩进（建议 4 个空格）是关键（Python 用缩进区分代码块）：

python

运行

示例：根据成绩判断等级

```

score = 85

if score >= 90: # 条件 1: 成绩≥90

    print("优秀")elif 80 <= score < 90: # 条件 2: 80≤成绩<90 (条件 1 不满足时判断)

    print("良好")elif 60 <= score < 80: # 条件 3: 60≤成绩<80 (前两个条件不满足时判断)

    print("及格")else: # 所有条件都不满足 (成绩<60)

    print("不及格")

# 运行结果: 良好

```

4.2 循环结构

4.2.1 for 循环（遍历序列）

用于遍历可迭代对象（如列表、元组、字符串等），执行固定次数的循环：

python

运行

示例 1: 遍历列表（水果列表）

```

fruits = ["苹果", "香蕉", "橙子"]for fruit in fruits: # 每次循环将列表中的一个元素赋值给 fruit

```

```

print(f'水果: {fruit}')# 输出: # 水果: 苹果# 水果: 香蕉# 水果: 橙子

```

示例 2: 遍历数字序列（用 range()函数生成连续整数）# range(起始值, 结束值, 步长): 包含起始值, 不包含结束值, 步长默认 1for i in range(1, 5): # 生成 1、2、3、4

```

print(f'数字: {i}')# 输出: # 数字: 1# 数字: 2# 数字: 3# 数字: 4

```

示例 3: 遍历字符串（每个字符）

```

text = "Python"for char in text:

```

```

print(char, end=" ") # 输出: P y t h o n

```

4.2.2 while 循环（条件控制）

只要条件为 `True`，就持续执行循环体，需注意避免“死循环”（需在循环体内修改条件变量）：

python

运行

示例：计数 1~3

```
count = 0 # 初始化计数器
```

```
while count < 3: # 条件：count<3 时执行循环
```

```
count += 1 # 计数器自增（关键：避免死循环）
```

```
print(f'计数: {count}')# 输出：# 计数：1# 计数：2# 计数：3
```

死循环示例（需避免）：条件永远为 `True`，且无修改条件的代码# while `True`:#
print("死循环")

4.2.3 循环控制：break 与 continue

`break`：立即终止当前循环，跳出循环体。

`continue`：跳过当前循环的剩余代码，直接进入下一次循环。

python

运行

示例：遍历 1~5，遇到 3 时终止循环（`break`）for i in range(1, 6):

```
if i == 3:
```

```
break # 终止循环
```

```
print(i)# 输出：1 2
```

示例：遍历 1~5，跳过 3（`continue`）for i in range(1, 6):

```
if i == 3:
```

```
continue # 跳过当前循环，进入下一次
```

```
print(i)# 输出：1 2 4 5
```

5. 核心数据结构（含元组）

数据结构用于存储和组织数据，Python 中常用的基础结构包括列表、元组、字典、集合，其中元组是“不可变序列”，列表是“可变序列”。

5.1 序列类型对比（列表 vs 元组）

类型	特点	定义方式	核心操作（增删改查）
列表 (list)	可变（可增删改元素）、有序	[] 或 list()	增: append()（末尾加）、insert()（指定位置加）； 删: pop()（删指定位置）、remove()（删指定值）； 改: list[索引] = 新值； 查: 索引 / 切片
元组 (tuple)	不可变（不能增删改元素）、有序	() 或 tuple() （小括号可省略）	增删改: 无（不可变）； 查: 索引 / 切片 / 拆包

5.2 元组核心操作

```
python
运行

# 1. 定义元组（小括号可省略，单个元素需加逗号，避免被当作普通括号）
point = (3, 4) # 坐标元组（两个元素）
color = "红", "绿", "蓝" # 颜色元组（小括号省略）
single_tuple = (5,) # 单个元素的元组（必须加逗号）

# 2. 索引访问（正向索引从 0 开始，反向索引从-1 开始）
print(point[0]) # 输出: 3（正向索引 0，第一个元素）
print(color[-1]) # 输出: 蓝（反向索引-1，最后一个元素）
```

3. 切片操作（获取子元组，语法：tuple[起始索引:结束索引:步长]，左闭右开）`print(color[1:])` # 输出：('绿','蓝')（从索引 1 到末尾）`print(point[:1])` # 输出：(3,)（从开头到索引 1，不包含索引 1）

4. 拆包特性（将元组元素赋值给多个变量，变量数量需与元组长度一致）

`x, y = point` # 拆包：x=3, y=4 `print(f'坐标: x={x}, y={y}')` # 输出：坐标：x=3, y=4

5. 不可变性（修改元组元素会报错，这是元组与列表的核心区别）`try`:

`point[0] = 5` # 尝试修改元组元素 `except TypeError as e`:

`print(e)` # 输出：'tuple' object does not support item assignment（元组不支持元素修改）

5.3 其他核心数据结构

5.3.1 字典（dict）：键值对存储

字典是“无序键值对集合”（Python 3.7 + 后有序），通过“键（key）”查找“值（value）”，适合存储关联数据（如用户信息）：

python

运行

1. 定义字典（用{}，键值对用冒号分隔，键必须唯一且不可变（如 str、int、tuple））

`user = {"name": "Alice", "age": 25, "is_student": True}`

2. 访问值（两种方式：dict[key] 或 dict.get(key)）`print(user["name"])`
输出：Alice（直接用键访问，键不存在会报错）`print(user.get("age"))` #
输出：25（用 get() 访问，键不存在返回 None，更安全）`print(user.get("gender", "未知"))` # 输出：未知（键不存在时返回默认值）

3. 修改/添加键值对

`user["age"] = 26` # 修改已有键的值

`user["gender"] = "女"` # 添加新键值对 `print(user)` # 输出：{'name': 'Alice', 'age': 26, 'is_student': True, 'gender': '女'}

4. 遍历字典（遍历键、值、键值对）`print("所有键: ", user.keys())` # 输出：所有键：dict_keys(['name', 'age', 'is_student', 'gender']) `print("所有值: ",`

```
user.values()) # 输出：所有值：dict_values(['Alice', 26, True, '女'])for k, v in
user.items():      # 遍历键值对

    print(f'{k}: {v}')
```

5.3.2 集合（set）：无序去重

集合是“无序、无重复元素”的集合，适合去重、交集 / 并集等运算：

python

运行

1. 定义集合（用 {} 或 set()，注意：空集合必须用 set()，{} 是空字典）

```
nums = {1, 2, 2, 3} # 有重复元素，定义后自动去重
```

```
empty_set = set() # 空集合
```

2. 去重效果 print(nums) # 输出：{1, 2, 3}（重复的 2 被去除）

3. 集合运算（交集、并集、差集）

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}print(set1 & set2) # 交集（共有的元素）： {3}print(set1
| set2) # 并集（所有元素，去重）： {1, 2, 3, 4, 5}print(set1 - set2) # 差集（set1
有但 set2 没有的元素）： {1, 2}
```

4. 添加/删除元素

```
nums.add(4) # 添加元素 4
```

```
nums.remove(2) # 删除元素 2（元素不存在会报错）print(nums) # 输出：
{1, 3, 4}
```

6. 函数定义与应用

函数是“封装好的可重复执行的代码块”，用于实现特定功能，减少代码冗余，提升可维护性。

6.1 函数基础语法

函数的定义用 def 关键字，语法结构：

python

运行

`def` 函数名(参数列表):

"""函数文档字符串（说明函数功能、参数、返回值，可选但推荐）"""

函数体（实现功能的代码）

`return` 返回值（可选，无返回值时默认返回 `None`）

示例：定义一个计算两数之和的函数：

python

运行

定义函数：计算两数之和 `def add_numbers(a, b):`

"""

计算两个数的和

参数：

a: 第一个数（int/float）

b: 第二个数（int/float）

返回值：

a + b 的结果（int/float）

"""

result = a + b

`return` result # 返回计算结果

调用函数：传入参数，获取返回值

sum_result = add_numbers(3, 5) # 传入 a=3, b=5, 返回值赋值给
sum_result
print(sum_result) # 输出：8

查看函数文档（用 `help()`）
help(add_numbers) # 输出函数的文档字符串
说明

6.2 函数参数类型

Python 函数支持多种参数类型，灵活适配不同场景：

6.2.1 位置参数（默认）

参数按定义顺序传入，数量和顺序必须与函数定义一致：

python

运行

```
def greet(name, age):  
    print(f'你好, {name}! 你{age}岁了。')  
  
# 位置参数调用: name="张三", age=20 (顺序对应)  
greet("张三", 20) # 输出: 你好, 张三! 你 20 岁了。
```

6.2.2 关键字参数

调用时指定参数名，无需严格按顺序，更清晰：

python

运行

关键字参数调用：顺序可换

```
greet(age=20, name="张三") # 输出: 你好, 张三! 你 20 岁了。
```

6.2.3 默认参数

定义函数时给参数设置默认值，调用时可省略该参数（默认参数需放在位置参数后面）：

python

运行

```
# 定义默认参数: msg 的默认值是"欢迎你"def greet(name, msg="欢迎你"):  
    print(f'{msg}, {name}!')  
  
# 调用 1: 省略默认参数, 使用默认值  
greet("张三") # 输出: 欢迎你, 张三!  
  
# 调用 2: 传入默认参数, 覆盖默认值  
greet("李四", "很高兴见到你") # 输出: 很高兴见到你, 李四!
```

6.2.4 可变参数 (*args)

用于接收不确定数量的位置参数，参数会被打包成一个元组：

python

运行

```
# 定义可变参数：*args 接收任意数量的位置参数
def sum_all(*args):
    """计算任意数量数字的和"""

    return sum(args) # sum()是 Python 内置函数，计算可迭代对象的和

# 调用：传入 1 个、2 个、3 个参数均可
print(sum_all(1)) # 输出：1
print(sum_all(1, 2)) # 输出：3
print(sum_all(1, 2, 3)) # 输出：6
```

6.2.5 关键字可变参数 (**kwargs)

用于接收不确定数量的关键字参数，参数会被打包成一个字典：

python

运行

```
# 定义关键字可变参数：**kwargs 接收任意数量的关键字参数
def print_info(**kwargs):
    """打印任意数量的关键字参数信息"""

    for k, v in kwargs.items():
        print(f'{k}: {v}')

# 调用：传入任意数量的关键字参数
print_info(name="Alice", age=25, gender="女") # 输出：# name: Alice# age: 25# gender: 女
```

7. 面向对象编程基础

面向对象编程（OOP）是一种编程思想，将数据（属性）和操作数据的方法封装在“类（Class）”中，通过“对象（Instance）”来调用，适合复杂项目的开发。

7.1 类与对象的基本概念

类(Class)：是“抽象的模板”，定义了对应的属性和方法（如“Person 类”定义了人的姓名、年龄属性，以及说话的方法）。

对象(Instance)：是“类的具体实例”，每个对象都有独立的属性值（如“张三”是“Person 类”的一个对象，姓名为“张三”，年龄为 20）。

7.2 定义类与创建对象

用 `class` 关键字定义类，`__init__` 是“构造方法”（初始化对象属性），`self` 代表“当前对象”：

```
python
运行

# 定义 Person 类 class Person:

# 构造方法：初始化对象的属性（name 和 age）

def __init__(self, name, age):

self.name = name # 实例属性：每个对象独有的属性

self.age = age # 实例属性


# 实例方法：对象的行为（需传入 self 参数）

def introduce(self):

"""返回对象的自我介绍"""

return f"我叫{self.name}，今年{self.age}岁"

# 创建对象（实例化类）：调用类名，传入__init__所需的参数（self 无需手动传）

person1 = Person("张三", 20) # person1 是 Person 类的一个对象

person2 = Person("李四", 18) # person2 是另一个对象

# 访问对象的属性 print(person1.name) # 输出：张三 print(person2.age) #
输出：18

# 调用对象的方法 print(person1.introduce()) # 输出：我叫张三，今年 20
岁 print(person2.introduce()) # 输出：我叫李四，今年 18 岁
```

7.3 继承特性（子类与父类）

继承允许“子类（Subclass）”复用“父类（Parent Class）”的属性和方法，并可添加自己的属性和方法，实现代码复用和扩展：

```
python

运行

# 父类：Person（已定义）# 子类：Student（继承自 Person）class
Student(Person):

# 子类的构造方法：需先调用父类的构造方法（用 super()）

def __init__(self, name, age, student_id):

    super().__init__(name, age) # 调用父类的__init__，初始化 name 和 age

    self.student_id = student_id # 子类新增的属性：学号


# 子类新增的方法：学习

def study(self, subject):

    return f'{self.name}（学号：{self.student_id}）正在学习{subject}'

# 创建子类对象

student1 = Student("李四", 18, "2025001")

# 调用父类的方法（子类继承了父类的 introduce()）
print(student1.introduce()) # 输出：我叫李四，今年 18 岁

# 调用子类的方法 print(student1.study("Python")) # 输出：李四（学号：
2025001）正在学习 Python

# 访问子类的属性 print(student1.student_id) # 输出：2025001
```

8. 文件操作（文本 / 二进制文件）

文件操作用于读取或写入外部文件（如文本文件、图片、视频等），Python 中用 `open()` 函数打开文件，推荐用 `with` 语句（自动关闭文件，避免资源泄漏）。

8.1 文本文件操作

文本文件（如 `.txt`、`.md`）存储的是字符数据，操作时需指定编码（如 `utf-8`，避免中文乱码）。

8.1.1 写入文本文件

python

运行

用 with 语句打开文件，模式为"w"（写入模式，覆盖原有内容；若文件不存在则创建）with open("test.txt", "w", encoding="utf-8") as f:

写入单行内容

f.write("Python 文件操作\n") # \n 是换行符

写入多行内容（可用 writelines()，传入字符串列表）

lines = ["文本文件读写示例\n", "这是第二行\n", "这是第三行"]

f.writelines(lines)

模式"a"（追加模式）：在文件末尾添加内容，不覆盖原有内容 with open("test.txt", "a", encoding="utf-8") as f:

f.write("追加的内容（不会覆盖原有内容）")

8.1.2 读取文本文件

常用读取方式有 3 种：read()（读全部内容）、readline()（读一行）、readlines()（读所有行，返回列表）：

python

运行

方式 1: read(): 读取全部内容 with open("test.txt", "r", encoding="utf-8") as f:

all_content = f.read()

print("全部内容: ")

print(all_content)

方式 2: readline(): 逐行读取（适合大文件，避免内存占用过多）print("\n逐行读取: ")with open("test.txt", "r", encoding="utf-8") as f:

while True:

line = f.readline() # 读一行

if not line: # 读到文件末尾，返回空字符串，退出循环

```

break

print(line.strip()) # strip()去除换行符和空格

# 方式 3: readlines(): 读取所有行, 返回列表 (每行是列表的一个元素)
with open("test.txt", "r", encoding="utf-8") as f:

    lines = f.readlines()

    print("\nreadlines()结果: ")

    for line in lines:

        print(line.strip())

```

8.2 二进制文件操作

二进制文件（如.jpg、.png、.mp4）存储的是字节数据，操作时模式需加 b（如 rb、wb），无需指定编码。

示例：复制一张图片（读取源图片的二进制数据，写入新图片）：

python

运行

```

# 二进制读 (rb) + 二进制写 (wb) with open("source.jpg", "rb") as f_read:
# 打开源图片, 只读二进制

with open("copy.jpg", "wb") as f_write: # 打开新图片, 只写二进制

# 读取源图片的所有二进制数据, 写入新图片

f_write.write(f_read.read())

print("图片复制成功!") # 运行后会生成 copy.jpg, 与 source.jpg 内容一致

```

8.3 常用文件打开模式

模式	说明	是否覆盖	是否创建新文件（不存在时）
r	只读（默认模式）	否	否（文件不存在报错）
w	只写（文本模式）	是	是

模式	说明	是否覆盖	是否创建新文件（不存在时）
a	追加（文本模式）	否	是
rb	只读（二进制模式）	否	否（文件不存在报错）
wb	只写（二进制模式）	是	是
ab	追加（二进制模式）	否	是
r+	读写（文本模式，从文件开头开始）	是	否（文件不存在报错）
a+	读写（文本模式，从文件末尾开始）	否	是

9. 模块与包（import 使用）

模块和包是 Python 组织代码的方式，用于实现代码的模块化和复用（避免代码都写在一个文件里）。

9.1 模块（Module）

模块：是一个.py 文件，包含 Python 代码（变量、函数、类等），可被其他文件导入使用。

导入模块的目的：复用已有代码（如 Python 标准库模块、第三方模块、自定义模块）。

9.1.1 导入模块的 4 种方式

```
python
运行
```

方式 1: 导入整个模块 (语法: `import 模块名`) `import math` # 导入 Python 标准库的 `math` 模块 (数学相关功能) # 使用模块中的函数/变量: 模块名.函数名/变量名 `print(math.sqrt(16))` # 输出: 4.0 (计算平方根) `print(math.pi)` # 输出: 3.141592653589793 (圆周率)

方式 2: 导入模块中的特定内容 (语法: `from 模块名 import 内容 1, 内容 2`) `from math import pi, pow` # 只导入 `math` 模块的 `pi` 和 `pow()` # 直接使用内容, 无需加模块名 `print(pi)` # 输出: 3.141592653589793 `print(pow(2, 3))` # 输出: 8.0 (计算 2 的 3 次方)

方式 3: 导入模块并指定别名 (语法: `import 模块名 as 别名`) `import random as rd` # 导入 `random` 模块 (随机数), 别名 `rd` (简化调用) # 使用别名调用 `print(rd.randint(1, 10))` # 输出: 1-10 之间的随机整数

方式 4: 导入模块的所有内容 (语法: `from 模块名 import *`) # ⚠ 不推荐: 可能导致命名冲突 (如不同模块有同名函数) `from math import *` `print(sin(0))` # 输出: 0.0 (计算正弦值, 无需加 `math.`)

9.1.2 自定义模块

自己编写的 `.py` 文件就是自定义模块, 可被其他文件导入。

示例:

1. 新建一个名为 `calc.py` 的文件 (自定义模块), 写入以下代码:

python

运行

`calc.py` (自定义模块) `def add(a, b):`

`"""计算两数之和"""`

`return a + b`

`def subtract(a, b):`

`"""计算两数之差"""`

`return a - b`

`PI = 3.1415926` # 模块中的变量

1. 在另一个文件 (如 `main.py`) 中导入并使用 `calc.py` 模块:

python

运行

```
# main.py# 导入自定义模块 calcimport calc

# 使用模块中的函数 print(calc.add(3, 5)) # 输出: 8print(calc.subtract(10,
4)) # 输出: 6

# 使用模块中的变量 print(calc.PI) # 输出: 3.1415926
```

9.2 包 (Package)

包：是“包含 `__init__.py` 文件的文件夹”，用于组织多个相关模块（如一个项目的“工具模块”“数据模块”可放在不同包中）。

`__init__.py`：是包的标识文件，可空（仅作标识），也可写入包的初始化代码（如导入包内的核心模块）。

9.2.1 包的目录结构

假设一个名为 `my_package` 的包，结构如下：

plaintext

```
my_package/          # 包目录
├── __init__.py      # 包标识文件（可空）
├── module1.py        # 模块 1：包含工具函数
└── module2.py        # 模块 2：包含数据处理类
```

9.2.2 导入包中的模块

python

运行

```
# 方式 1：从包中导入模块（语法：from 包名 import 模块名）from
my_package import module1# 使用模块中的函数

module1.tool_func() # 假设 module1 中有 tool_func()函数

# 方式 2：从包的模块中导入特定内容（语法：from 包名.模块名 import 内
容）from my_package.module2 import DataProcessor# 使用导入的类

dp = DataProcessor() # 实例化 DataProcessor 类
```

```
dp.process_data()    # 调用类的方法

# 方式 3: 导入包并指定别名 import my_package.module1 as m1

m1.tool_func() # 用别名调用
```

9.2.3 __init__.py 的作用

若在 my_package/__init__.py 中写入以下代码:

python

运行

```
# my_package/__init__.py from .module1 import tool_func # 从当前包的
module1 导入 tool_func from .module2 import DataProcessor # 从当前包的
module2 导入 DataProcessor
```

则可直接从包中导入内容, 无需写模块名:

python

运行

```
from my_package import tool_func, DataProcessor
```

```
tool_func() # 直接调用
```

```
dp = DataProcessor() # 直接实例化
```

10. 异常处理 (try-except)

异常是程序运行时发生的错误 (如除以 0、文件不存在、类型错误等), 若不处理会导致程序崩溃。异常处理用 try-except 语句捕获错误, 让程序优雅地继续运行。

10.1 异常处理基础语法

python

运行

```
try:
```

```
# 可能发生异常的代码块 (核心逻辑)
```

代码 1

代码 2 `except` 异常类型 1:

捕获到“异常类型 1”时执行的代码（处理逻辑）

处理代码 1 `except` 异常类型 2 `as e`:

捕获到“异常类型 2”时执行的代码，`e` 是异常对象（包含错误信息）

处理代码 2（可通过 `e` 获取错误详情）`else`:

可选：`try` 块中无异常时执行的代码

无异常代码 `finally`:

可选：无论是否有异常，都会执行的代码（如释放资源、关闭文件）

必执行代码

示例：处理“输入非整数”的异常：

python

运行

`try`:

可能出错的代码：用户输入非整数时，`int()`会触发 `ValueError`

```
num = int(input("请输入整数: "))
```

```
print(f'你输入的整数是: {num}')except ValueError:
```

捕获 `ValueError`，提示用户

```
print("错误: 请输入有效的整数! (如 123、456)")except Exception as e:
```

捕获所有其他异常（兜底，避免未预料的错误导致程序崩溃）

```
print(f'未知错误: {e}')else:
```

无异常时执行

```
print("输入成功, 无异常!")finally:
```

无论是否有异常，都会执行

```
print("程序执行完毕, 感谢使用!")
```

10.2 常见异常类型

Python 内置了多种异常类型，以下是开发中常见的几种：

异常类型	说明	示例场景
ValueError	值错误（数据类型正确，但值不符合要求）	<code>int("abc")</code> （字符串是 <code>str</code> 类型，但无法转整数）
TypeError	类型错误（操作的对象类型不符合要求）	<code>1 + "2"</code> （整数和字符串无法相加）
FileNotFoundError	文件未找到错误（打开不存在的文件）	<code>open("nonexist.txt", "r")</code> （文件不存在）
IndexError	索引越界错误（访问列表 / 元组的无效索引）	<code>list1 = [1,2]; list1[3]</code> （列表只有 2 个元素，索引 3 无效）
ZeroDivisionError	除以零错误（除数为 0）	<code>10 / 0</code>

10.3 自定义异常

当 Python 内置异常无法满足需求时，可自定义异常类（继承 `Exception` 类）：

python

运行

```
# 自定义异常类（继承 Exception）class CustomError(Exception):
```

```
    """自定义异常：用于表示特定业务场景的错误（如“年龄小于 0”）"""
```

```
    def __init__(self, message):
```

```
        self.message = message # 异常信息
```

```
# 可选：重写 __str__ 方法，自定义异常的打印格式
```

```
def __str__(self):  
    return f"CustomError: {self.message}"  
  
# 使用自定义异常（用 raise 语句主动触发）try:  
age = int(input("请输入年龄: "))  
  
if age < 0:  
    # 主动触发自定义异常  
    raise CustomError(f"年龄不能为负数（你输入的是{age}）")  
  
print(f"年龄: {age}")except CustomError as e:  
    # 捕获自定义异常  
    print(e) # 输出: CustomError: 年龄不能为负数（你输入的是-5）（若输入  
-5）
```

11. 正则表达式基础

正则表达式（Regex）是“用于匹配字符串模式的工具”，可快速实现字符串的查找、替换、验证等功能（如验证邮箱格式、提取手机号）。Python 中用 `re` 模块操作正则表达式。

11.1 核心元字符与量词

正则表达式通过“元字符”（特殊符号）和“量词”（匹配次数）定义匹配模式，以下是常用的基础符号：

类别	符号	说明	示例
元字符	.	匹配任意单个字符（除换行符 \n）	<code>a.b</code> 匹配 "aab"、 "acb"、"alb"
	^	匹配字符串的开头	<code>^Python</code> 匹配 "Python 编程"

类别	符号	说明	示例
量词	\$	匹配字符串的结尾	end\$ 匹配 "程序 end"
	[]	匹配括号内的任意一个字符	[abc] 匹配 "a"、"b"、 "c"
	\d	匹配数字 (0-9)，等价于 [0-9]	\d{3} 匹配 "123"、 "456"
	\w	匹配字母、数字、下划线，等价于 [a-zA-Z0-9_]	\w+ 匹配 "name123"、 "user_name"
	\s	匹配空白字符（空格、制表符 \t、换行符 \n）	a\s b 匹配 "a b"、 "a\tb"
	*	匹配前面的元素 0 次或多次	a* 匹配 ""、"a"、"aa"、 "aaa"
	+	匹配前面的元素 1 次或多次	a+ 匹配 "a"、"aa"、 "aaa"
	?	匹配前面的元素 0 次或 1 次（可选）	a? 匹配 ""、"a"
	{n}	匹配前面的元素恰好 n 次	a{2} 匹配 "aa"
	{n,}	匹配前面的元素至少 n 次	a{2,} 匹配 "aa"、 "aaa"
	{n,m}	匹配前面的元素 $n \sim m$ 次	a{2,3} 匹配 "aa"、

类别	符号	说明	示例
			"aaa"

11.2 re 模块常用函数

re 模块提供了多个操作正则表达式的函数，以下是最常用的 4 个：

11.2.1 re.match(pattern, string)

功能：从字符串**开头**匹配模式，若开头不匹配则返回 None。

返回值：匹配成功返回 Match 对象（可通过 group() 获取匹配内容），失败返回 None。

示例：

python

运行

```
import re
```

```
# 匹配以"Python"开头的字符串
```

```
pattern = r"Python" # r 表示“原始字符串”，避免转义字符（如\d无需写成\\d）
```

```
string1 = "Python 编程基础"
```

```
string2 = "编程基础 Python"
```

```
result1 = re.match(pattern, string1)
```

```
result2 = re.match(pattern, string2)
```

```
if result1:
```

```
    print("string1 匹配成功: ", result1.group()) # 输出: string1 匹配成功:
```

```
Python
```

```
else:
```

```
    print("string1 匹配失败")
```

```
if result2:
```

```
print("string2 匹配成功: ", result2.group())else:
print("string2 匹配失败") # 输出:string2 匹配失败(string2 开头不是 Python)
```

11.2.2 re.search(pattern, string)

功能：在整个字符串中查找**第一个**匹配模式的子串，不限制开头。

区别于 match: match 只匹配开头, search 匹配整个字符串的第一个符合项。

示例：

python

运行

在字符串中查找第一个数字序列

```
pattern = r"\d+" # \d+: 匹配 1 个或多个数字
```

```
string = "年龄: 25, 身高: 175cm"
```

```
result = re.search(pattern, string)if result:
```

```
print("第一个匹配的数字: ", result.group()) # 输出: 第一个匹配的数字:
```

25

11.2.3 re.findall(pattern, string)

功能：在整个字符串中查找**所有**匹配模式的子串，返回一个列表（包含所有匹配项）。

示例：

python

运行

查找字符串中所有的数字序列

```
pattern = r"\d+"
```

```
string = "年龄: 25, 身高: 175cm, 体重: 60kg"
```

```
result = re.findall(pattern, string)print("所有匹配的数字: ", result) #
输出: 所有匹配的数字: ['25', '175', '60']
```

11.2.4 re.sub(pattern, repl, string)

功能：将字符串中所有匹配模式的子串替换为 repl（替换内容），返回替换后的字符串。

示例：

python

运行

将字符串中的数字替换为"XX"

```
pattern = r"\d+"
```

```
string = "年龄：25，身高：175cm"
```

```
repl = "XX"
```

```
result = re.sub(pattern, repl, string)print("替换后：", result) # 输出：  
替换后：年龄：XX，身高：XXcm
```

11.3 实用场景示例：验证邮箱格式

邮箱格式的正则表达式模式（简化版）：

```
^[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+\.[a-zA-Z]{2,}$
```

^[a-zA-Z0-9_-]+：邮箱用户名（字母、数字、下划线、减号，1 个或多个字符）

@：必须包含 @符号

[a-zA-Z0-9_-]+：邮箱域名（如 gmail、163）

[a-zA-Z]{2,}：域名后缀（如.com、.cn，2 个或多个字母）

示例：

python

运行

```
import re
```

```
def is_valid_email(email):
```

```
    """验证邮箱格式是否正确"""
```

```
    pattern = r"^[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+\.[a-zA-Z]{2,}$"
```

```

# re.match(): 从开头到结尾完全匹配（因为 pattern 有^和$）

iare.match(pattern, email):

return True

else:

return False

# 测试

emails = ["test@example.com", "test123_@163.cn", "test@.com",
"test@com"]for email in emails:

    ibis_valid_email(email):

    print(f'{email}: 邮箱格式正确')

    else:

    print(f'{email}: 邮箱格式错误')

# 输出: # test@example.com: 邮箱格式正确# test123_@163.cn: 邮箱格式
正确# test@.com: 邮箱格式错误（域名部分为空）# test@com: 邮箱格式错误（无
后缀，如.com）

```

12. 列表推导式与字典推导式

推导式是 Python 的“语法糖”，用简洁的一行代码生成列表、字典等数据结构，替代繁琐的 for 循环，提升代码效率和可读性。

12.1 列表推导式

12.1.1 基础语法

python

运行

语法: [表达式 for 变量 in 可迭代对象]# 功能: 遍历可迭代对象，对每个变量执行“表达式”，将结果组成新列表

示例 1: 生成 1~10 的平方列表（替代 for 循环）# 普通 for 循环写法:

```

squares = []for x in range(1, 11):

squares.append(x * x)

```

列表推导式写法（一行搞定）：

```
squares = [x * x for x in range(1, 11)]print(squares) # 输出：[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

12.1.2 带条件筛选的列表推导式

python

运行

语法：[表达式 for 变量 in 可迭代对象 if 条件]# 功能：遍历可迭代对象，只对满足“条件”的变量执行“表达式”，组成新列表

示例：生成 1~10 的偶数列表# 普通 for 循环写法：

```
evens = []for x in range(1, 11):
```

```
    if x % 2 == 0:
```

```
        evens.append(x)
```

列表推导式写法：

```
evens = [x for x in range(1, 11) if x % 2 == 0]print(evens) # 输出：[2, 4, 6, 8, 10]
```

12.1.3 嵌套列表推导式

python

运行

语法：[表达式 for 外层变量 in 外层可迭代对象 for 内层变量 in 内层可迭代对象]# 功能：嵌套遍历，相当于外层 for 循环嵌套内层 for 循环

示例：将二维列表（矩阵）转为一维列表

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

普通 for 循环写法：

```
flatten = []for row in matrix:
```

```
    for x in row:
```

```
        flatten.append(x)
```

嵌套列表推导式写法：

```
flatten = [x for row in matrix for x in row]print(flatten) # 输出: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

12.2 字典推导式

12.2.1 基础语法

python

运行

语法: {键表达式: 值表达式 for 变量 in 可迭代对象}# 功能: 遍历可迭代对象, 生成键值对, 组成新字典

示例 1: 生成键为 1~5, 值为对应平方的字典# 普通 for 循环写法:

```
square_dict = {}for x in range(1, 6):
```

```
square_dict[x] = x * x
```

字典推导式写法:

```
square_dict = {x: x * x for x in range(1, 6)}print(square_dict) # 输出: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

12.2.2 带条件筛选的字典推导式

python

运行

语法: {键表达式: 值表达式 for 变量 in 可迭代对象 if 条件}# 功能: 只对满足“条件”的变量生成键值对

示例: 筛选字典中值大于 10 的键值对

```
square_dict = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

普通 for 循环写法:

```
filtered_dict = {}for k, v in square_dict.items():
```

```
if v > 10:
```

```
filtered_dict[k] = v
```

字典推导式写法:

```
filtered_dict = {k: v for k, v in square_dict.items() if v > 10}print(filtered_dict) # 输出: {4: 16, 5: 25}
```

12.2.3 用两个可迭代对象生成字典

若有两个可迭代对象（如键列表和值列表），可通过 `zip()` 函数配对后生成字典：

```
python
运行
# 键列表和值列表
keys = ["name", "age", "gender"]
values = ["Alice", 25, "女"]
# 字典推导式写法（用 zip() 配对键和值）
user_dict = {k: v for k, v in zip(keys, values)}print(user_dict) # 输出:
{'name': 'Alice', 'age': 25, 'gender': '女'}
```

13. 综合实战案例：学生信息管理系统

整合前面所学的所有知识点（面向对象、文件操作、异常处理、正则表达式、列表推导式等），实现一个简单的学生信息管理系统，功能包括：

1. 加载历史学生信息（从文件读取）；
2. 添加学生信息（验证学号格式、保存到文件）；
3. 筛选特定条件的学生（用列表推导式）；
4. 处理异常（如文件不存在、学号格式错误）。

13.1 完整代码

```
python
运行
import re # 导入正则表达式模块，用于验证学号格式 from datetime import
datetime # 导入 datetime 模块，可用于记录操作时间（本案例暂用）
# 1. 定义 Student 类（继承自之前的 Person 类，新增学号属性）class Person:
def __init__(self, name, age):
```

```
self.name = name
```

```
self.age = age
```

```
def introduce(self):
```

```
return f"我叫{self.name}, 今年{self.age}岁"
```

```
class Student(Person):
```

```
def __init__(self, name, age, student_id):
```

```
super().__init__(name, age)
```

```
self.student_id = student_id # 学号属性
```

```
def introduce(self):
```

```
# 重写父类的 introduce(), 增加学号信息
```

```
return f"我叫{self.name}, 今年{self.age}岁, 学号: {self.student_id}"
```

2. 定义 StudentManager 类（管理学生信息的核心逻辑）
class StudentManager:

```
def __init__(self):
```

```
self.students = [] # 存储所有学生对象的列表
```

```
# 初始化时加载历史学生信息（从文件读取）
```

```
self.load_students_from_file()
```

```
def load_students_from_file(self):
```

```
"""从文件中加载历史学生信息"""
```

```
try:
```

```
# 读取 students.txt 文件（若存在）
```

```
with open("students.txt", "r", encoding="utf-8") as f:
```

```
    for line in f.readlines():
```

```
        # 每行格式: 姓名,年龄,学号（需处理换行符）
```

```
        line = line.strip()
```

```
        if not line: # 跳过空行
```

```
            continue
```

```

name, age_str, student_id = line.split(",")

        # 将年龄转为整数，创建 Student 对象并添加到列表
student = Student(name, int(age_str), student_id)

self.students.append(student)

print(f"✅ 成功加载历史信息，共{len(self.students)}名学生")

except FileNotFoundError:

    # 文件不存在时，提示用户（不报错，后续添加学生时会创建文件）
    print(f"❗ 未找到历史学生文件，将创建新文件")

    except Exception as e:

        # 处理其他未知异常（如文件格式错误）

        print(f"❌ 加载学生信息失败：{e}")


def save_student_to_file(self, student):

    """将新添加的学生信息保存到文件（追加模式）"""

    try:

        with open("students.txt", "a", encoding="utf-8") as f:

            # 按“姓名,年龄,学号”格式写入
            f.write(f"{student.name},{student.age},{student.student_id}\n")

        print(f"✅ 学生信息已保存到文件")

        except Exception as e:

            print(f"❌ 保存学生信息失败：{e}")


def is_valid_student_id(self, student_id):

    """用正则表达式验证学号格式（要求：6 位数字）"""

    pattern = r"^\d{6}$" # 6 位数字的正则模式

    return bool(re.match(pattern, student_id))

```

```

def add_student(self, name, age, student_id):
    """添加学生信息（包含格式验证、异常处理）"""

    # 1. 验证学号格式

    if not self.is_valid_student_id(student_id):

        print(f"✗ 学号格式错误！学号必须是 6 位数字（你输入的是：{student_id}）")

    ")

    return

    # 2. 验证年龄是否为正数

    if age <= 0:

        print(f"✗ 年龄错误！年龄必须是正数（你输入的是：{age}）")

    return

    # 3. 验证学号是否已存在

    for stu in self.students:

        if stu.student_id == student_id:

            print(f"✗ 学号已存在！学号 {student_id} 对应的学生是： {stu.name}")

            return

    # 4. 创建 Student 对象并添加到列表

    new_student = Student(name, age, student_id)

    self.students.append(new_student)

    print(f"☑ 添加成功！ {new_student.introduce()}")

    # 5. 保存到文件

    self.save_student_to_file(new_student)

```

```

def filter_students(self, min_age):
    """用列表推导式筛选年龄≥min_age 的学生"""
    filtered = [stu for stu in self.students if stu.age >= min_age]
    return filtered

# 3. 主程序（运行学生信息管理系统）if __name__ == "__main__":
print("=" * 50)

print("          学生信息管理系统          ")

print("=" * 50)

# 创建 StudentManager 对象（自动加载历史信息）
manager = StudentManager()

# 添加学生（示例）
print("\n" + "-" * 30)
print("开始添加学生：")

manager.add_student("张三", 20, "202501") # 学号 202501（6 位，正确）
manager.add_student("李四", 18, "202502") # 学号 202502（6 位，正确）
manager.add_student("王五", 19, "2025a3") # 学号含字母（错误）
manager.add_student("赵六", 21, "2025045") # 学号 7 位（错误）

# 筛选 18 岁以上的学生（用列表推导式）
print("\n" + "-" * 30)
print("18 岁以上的学生：")

adults = manager.filter_students(min_age=18)

if adults:
    for idx, stu in enumerate(adults, 1): # enumerate()带索引
        print(f'{idx}. {stu.introduce()}')
    else:
        print("未找到 18 岁以上的学生")

```

```
print("\n" + "=" * 50)

print("系统运行结束")

print("=" * 50)
```

13.2 代码说明与运行结果

13.2.1 核心知识点整合

面向对象：Person 父类、Student 子类、StudentManager 管理类，封装属性和方法。

文件操作：load_students_from_file() 读取历史信息，save_student_to_file() 保存新信息。

异常处理：捕获 FileNotFoundError（文件不存在）、未知异常，避免程序崩溃。

正则表达式：is_valid_student_id() 验证学号是否为 6 位数字。

列表推导式：filter_students() 用推导式筛选符合条件的学生。

枚举函数：enumerate(adults, 1) 为学生列表添加从 1 开始的索引，便于展示。

13.2.2 运行结果

```
plaintext
=====

学生信息管理系统

=====

[i] 未找到历史学生文件，将创建新文件

-----

开始添加学生：

[✓] 添加成功！我叫张三，今年 20 岁，学号：202501

[✓] 学生信息已保存到文件
```

- ☒ 添加成功！我叫李四，今年 18 岁，学号：202502
- ☒ 学生信息已保存到文件
- ☒ 学号格式错误！学号必须是 6 位数字（你输入的是：2025a3）
- ☒ 学号格式错误！学号必须是 6 位数字（你输入的是：2025045）

-
- 18 岁以上的学生：
1. 我叫张三，今年 20 岁，学号：202501
 2. 我叫李四，今年 18 岁，学号：202502

=====

系统运行结束

=====

同时，会生成 `students.txt` 文件，内容如下：

plaintext

张三,20,202501

李四,18,202502

14. 知识点总览与拓展

14.1 核心知识点全覆盖（分层梳理）

层级	知识点	作用
基础层	变量、数据类型、输入输出、流程控制	搭建 Python 编程的基础框架
数据结构层	列表、元组、字典、集合	高效存储和组织数据
代码复用层	函数、模块、包	减少代码冗余，提升可维护性

层级	知识点	作用
高级编程层	面向对象、异常处理	应对复杂项目，提升代码稳定性和扩展性
工具层	文件操作、正则表达式、推导式	实现外部交互（文件）、高效处理字符串和数据

14.2 拓展学习方向

掌握基础后，可根据兴趣和需求深入以下领域：

1. **高级数据结构：**学习 `collections` 模块（如 `deque` 队列、`OrderedDict` 有序字典、`defaultdict` 默认字典），应对复杂数据场景。

2. **网络编程：**用 `requests` 模块发送 HTTP 请求（爬取网页数据、调用 API），用 `socket` 模块实现 TCP/UDP 通信。

3. **数据处理与可视化：**

`numpy`：处理数值计算（矩阵运算、数组操作）；

`pandas`：处理表格数据（读取 Excel/Csv、数据清洗、统计分析）；

`matplotlib/seaborn`：绘制折线图、柱状图、饼图等可视化图表。

4. **Web 开发：**学习 `Django`（全能型框架）或 `Flask`（轻量型框架），开发网站和 Web 应用。

5. **自动化脚本：**用 `selenium` 实现网页自动化（如自动登录、数据爬取），用 `os/shutil` 模块实现文件批量处理。

6. 推荐学习资源

官方文档：[Python 官方文档](#)（最权威，包含所有语法和模块说明）。

书籍：

《Python 编程》

《Python 编程快速上手》

《Python 背记手册》

